

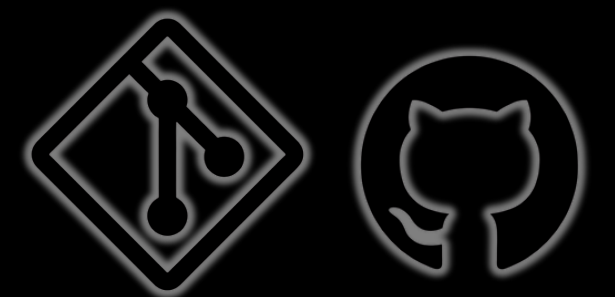
WORKSHOP

GIT E GITHUB

O QUE É O GIT?

O Git é um sistema de controle de versão que funciona como um histórico para o código, o que permite rastrear alterações, salvar estados do projeto e colaborar com a segurança.

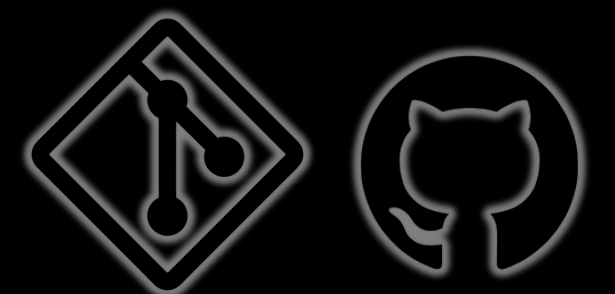
- Histórico de Alterações
- Trabalho em Equipe
- Controle Distribuído



O QUE É O GITHUB?

O GitHub é uma plataforma de hospedagem baseada em nuvem que utiliza o sistema Git para gerenciar e armazenar código-fonte.

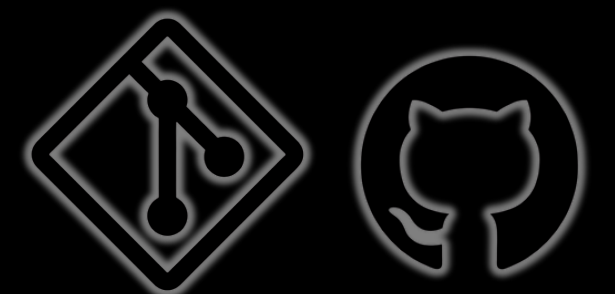
- Hospedagem de Repositórios
- Colaboração em Equipe
- Portfólio Profissional
- Recursos Extras



QUAL A DIFERENÇA?

A diferença é que o Git é a ferramenta e o GitHub é o lugar onde você guarda o que você fez com essa ferramenta.

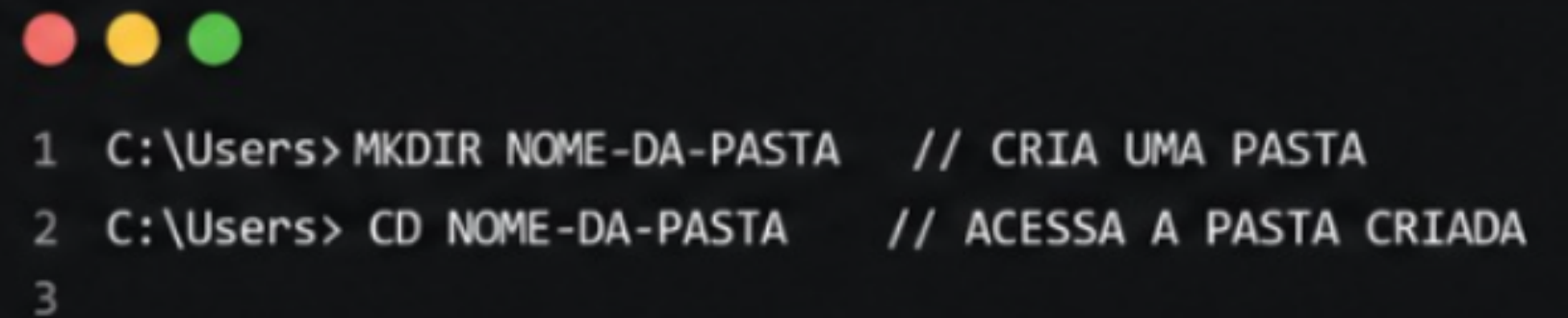
O Git é como Word, onde você escreve e salva as versões. E o GitHub é como o Google Drive, onde você sobe o arquivo para compartilhar e salvar na nuvem.



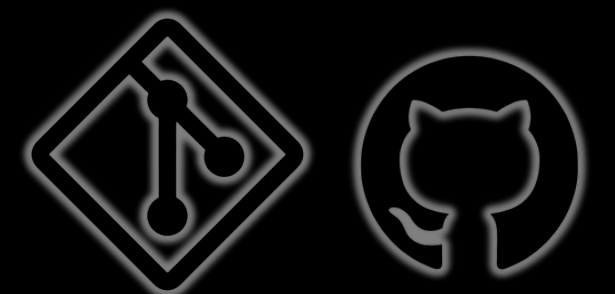
CRIANDO A PASTA

Abrir o CMD (Terminal).

- `mkdir nome-do-projeto`: Cria a pasta.
- `cd nome-do-projeto`: Entra na pasta.



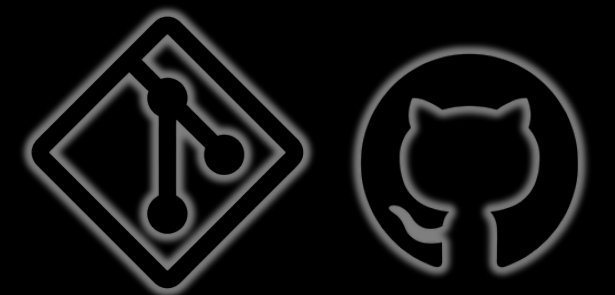
```
1 C:\Users> MKDIR NOME-DA-PASTA // CRIA UMA PASTA
2 C:\Users> CD NOME-DA-PASTA // ACESSA A PASTA CRIADA
3
```



PASTA COMUM VS. REPOSITÓRIO

Uma pasta comum é apenas um lugar para guardar arquivos.

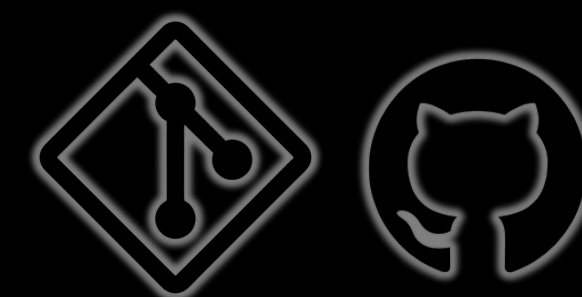
Um repositório contém um histórico de versões escondido na pasta .git.



INICIALIZANDO O REPOSITÓRIO LOCAL

Para inicializar o repositório local é necessário usar o comando **git init**.

```
1 C:\Users\NOME-DA-PASTA> GIT INIT // INICIA O REPOSITÓRIO LOCAL
2 C:\Users\NOME-DA-PASTA>
3
```

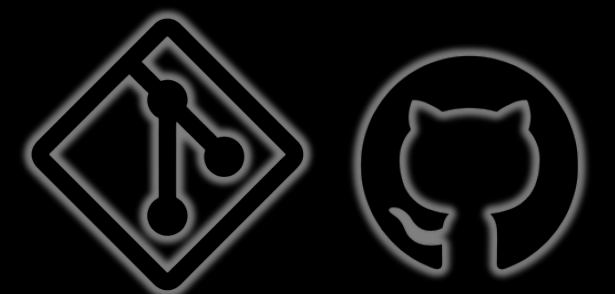


CRIANDO O PROJETO

Para criar o projeto, é necessário usar o comando **code**.



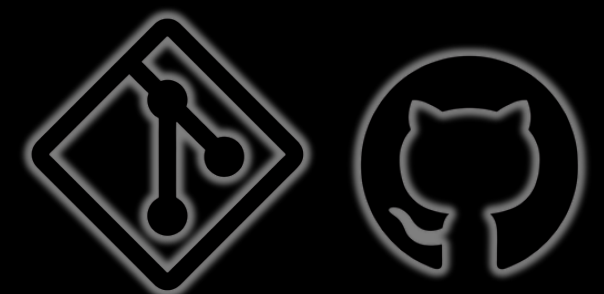
```
1 C:\Users\NOME-DA-PASTA> CODE . // ABRE O VS CODE NA PASTA ATUAL
2 C:\Users\NOME-DA-PASTA>
3
```



FLUXO DE COMMIT

Os 3 estados do Git:

- **Untracked (Não monitorado):** É o estado inicial de qualquer arquivo novo.
- **Staged (Preparado):** É a área de transição (Staging Area).
- **Committed (Confirmado):** O arquivo está salvo com segurança no seu banco de dados local.

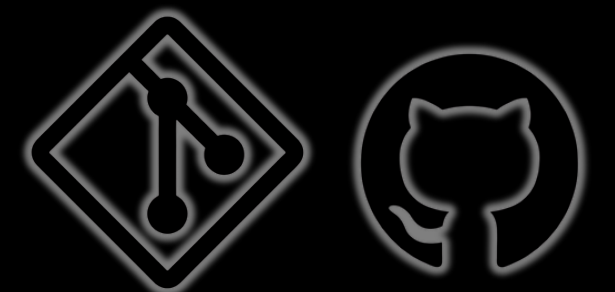


IDENTIFICANDO O USUARIO

Antes de começar os commits, o Git precisa saber quem você é. Essas informações aparecerão no histórico do projeto para identificar o autor de cada mudança.

- Definir Nome: `git config --global user.name "Seu Nome"`
- Definir E-mail: `git config --global user.email "seuemail@exemplo.com"`

```
1 C:\Users\NOME-DA-PASTA> GIT CONFIG --GLOBAL USER.NAME "SEU USUARIO"  
2 C:\Users\NOME-DA-PASTA> GIT CONFIG --GLOBAL USER.EMAIL "SEU EMAIL"  
3 C:\Users\NOME-DA-PASTA>  
4
```



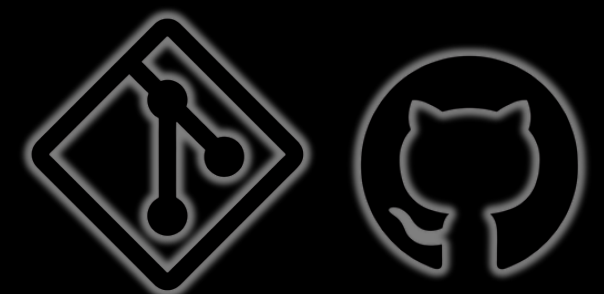
ADICIONANDO OS ARQUIVOS NO **GIT**

Para preparar e adicionar os arquivos no repositório, é necessário usar o comando **git add**.

E, para vermos os status dos arquivos, usamos o comando **git status**.

```
1 C:\Users\NOME-DA-PASTA> GIT STATUS // MOSTRA OS ARQUIVOS UNTRACKED
2 C:\Users\NOME-DA-PASTA>
3
```

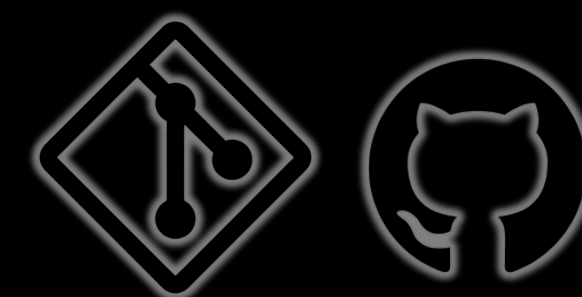
```
1 C:\Users\NOME-DA-PASTA> GIT ADD . // MOVE PARA STAGED
2 C:\Users\NOME-DA-PASTA>
```



COMO FAZER O COMMIT

Para “comitar”, é necessário usar o comando **git commit -am “Mensagem do usuário”**.

```
1 C:\Users\NOME-DA-PASTA> GIT COMMIT -AM "MENSAGEM DO USUARIO"  
2 C:\Users\NOME-DA-PASTA>  
3
```



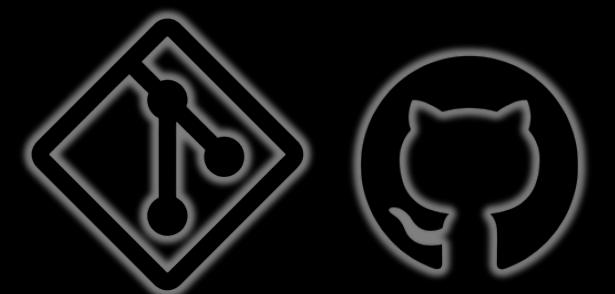
HISTÓRICO DE COMMITS

O comando **git log** funciona como um "diário" do seu projeto, listando todas as versões salvas em ordem cronológica inversa.

O comando **git diff** permite visualizar a diferença de uma alteração feita no arquivo antes dela ser commitada

```
1 C:\Users\NOME-DA-PASTA> GIT LOG // MOSTRA O HISTÓRICO DE COMMITS
2 C:\Users\NOME-DA-PASTA>
```

```
1 C:\Users\NOME-DA-PASTA> GIT DIFF // MOSTRA AS ALTERAÇÕES FEITAS
2 C:\Users\NOME-DA-PASTA>
```

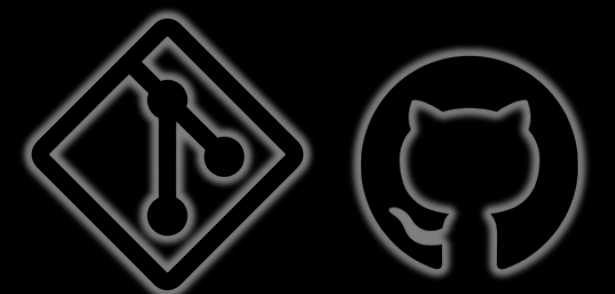


COMO CONECTAR AO REPOSITÓRIO REMOTO

O comando **git remote add origin** 'link' conecta seu projeto local ao repositório no GitHub.

Para verificar, usamos o comando **git remote -v**.

```
1 C:\Users\NOME-DA-PASTA> GIT REMOTE ADD ORIGIN "LINK DO REPOSITORIO" // CONECTA AO REPOSITÓRIO REMOTO
2 C:\Users\NOME-DA-PASTA> GIT REMOTE -V // VERIFICA O REPOSITÓRIO REMOTO
3 C:\Users\NOME-DA-PASTA>
```

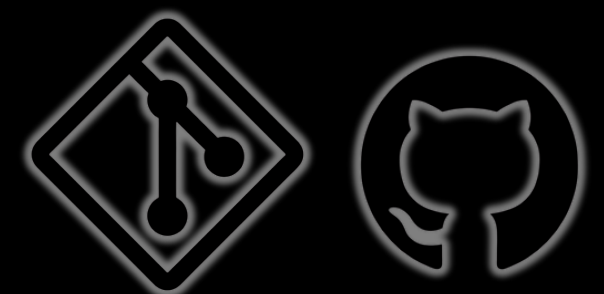


ATENTICAÇÃO COM TOKEN (PAT)

A autenticação com o Token (PAT) funciona como uma chave mestra digital que dá permissão ao computador para gerenciar seus projetos no GitHub.

- Perfil > Settings > Developer settings > Personal access tokens > Tokens (classic) > Clique em Generate new token (classic).
- Configurar: Dê um Nome (ex: Workshop). Marque a opção repo (obrigatório).
- Salvar: Clique em Generate token no fim da página.

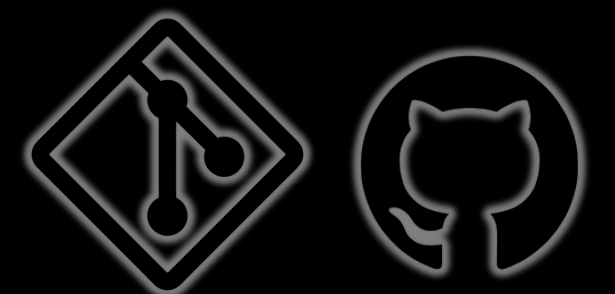
Dica de ouro: se você sair da página sem copiar, terá que gerar outro, pois ele nunca mais será exibido.



SUBINDO O PROJETO NO GITHUB

O comando **git push** é o ato de "empurrar" seus commits locais para o servidor online.

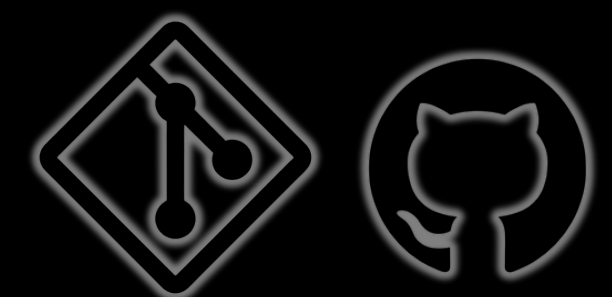
```
1 C:\Users\NOME-DA-PASTA> GIT PUSH ORIGIN MAIN // ENVIA OS ARQUIVOS
2 C:\Users\NOME-DA-PASTA>
3
```



SINCRONIZANDO ALTERAÇÕES

O comando **git pull** faz o caminho inverso: ele "puxa" as atualizações que estão no GitHub para a sua máquina local.

```
1 C:\Users\NOME-DA-PASTA> GIT PULL ORIGIN MAIN // BAIXA AS MUDANÇAS FEITAS NO REPOSITÓRIO REMOTO
2 C:\Users\NOME-DA-PASTA>
```



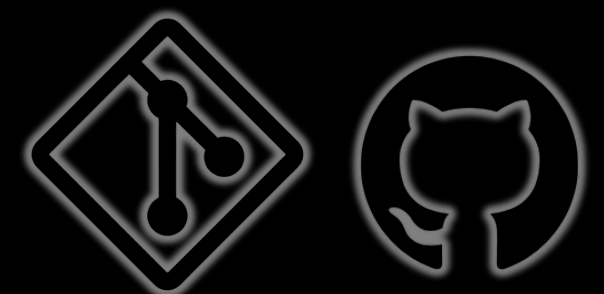
COMO COPIAR SEU REPOSITÓRIO?

Para clonar um repositório remoto, usamos **git clone 'link'**.

Depois, acessamos a pasta com **cd "nome da pasta"** e abrimos no VS Code com **code .**



```
1 C:\Users\NOME-DA-PASTA> GIT CLONE <URL DO REPOSITÓRIO> // CLONA O REPOSITÓRIO
2 C:\Users\NOME-DA-PASTA> CD PASTA-DO-REPOSITORIO // ENTRA NA PASTA DO REPOSITÓRIO
3 C:\Users\NOME-DA-PASTA\PASTA-DO-REPOSITORIO> CODE . // ABRE NO VS CODE
4 C:\Users\NOME-DA-PASTA\PASTA-DO-REPOSITORIO>
```



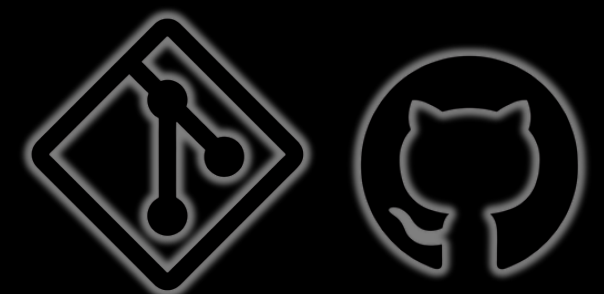
O QUE É UMA BRANCH?

As branches permitem trabalhar em versões diferentes do projeto ao mesmo tempo, sem afetar o código principal.

A branch padrão é a main, onde nasce o projeto e normalmente não se faz alterações diretamente.

Elas são essenciais para desenvolver novas funcionalidades, corrigir erros e testar ideias em paralelo.

```
1 C:\Users\NOME-DA-PASTA> GIT CLONE <URL DO REPOSITÓRIO> // CLONA O REPOSITÓRIO
2 C:\Users\NOME-DA-PASTA> CD PASTA-DO-REPOSITORIO // ENTRA NA PASTA DO REPOSITÓRIO
3 C:\Users\NOME-DA-PASTA\PASTA-DO-REPOSITORIO> CODE . // ABRE NO VS CODE
4 C:\Users\NOME-DA-PASTA\PASTA-DO-REPOSITORIO>
```

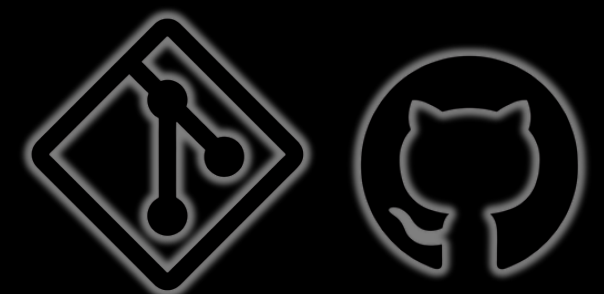


RAMIFICAÇÕES

Uma branch (ramificação) é como uma cópia paralela do seu projeto onde você pode fazer alterações sem afetar o código principal.



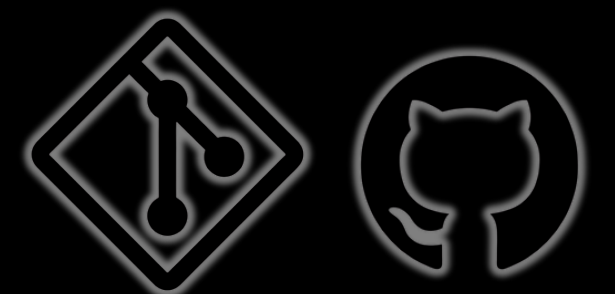
- 1 // BRANCHES PERMITEM TRABALHAR EM DIFERENTES VERSÕES DO PROJETO
- 2 // A BRANCH PADRÃO É MAIN OU MASTER (VERSÃO PRINCIPAL)
- 3 // EVITE FAZER COMMITS DIRETAMENTE NELA

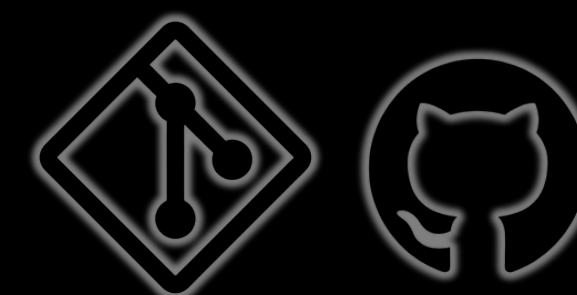


QUIZ

QRCode:

Código da sala:





OBRIGADO!

